Java DSA Problem-Solving Cheat Sheet

A compact workflow for breaking coding problems into solvable steps, choosing the right pattern, and writing cleaner Java solutions.

CORE LOOP

Understand. Pattern match. Code. Verify.

Input -> Output | Pattern Match | Brute Force First | Optimize by Constraint | Test Edge Cases

1. BREAKDOWN STEPS

1. Restate the problem in one sentence.
2. Write exact input and output types.
3. Check constraints: size, ordering, duplicates, ranges.
4. Trace one small example by hand.
5. Write a brute-force idea first.
6. Look for a known pattern or better structure.
7. Estimate time and space before coding.
8. Test normal and edge cases after coding.

2. KEY QUESTIONS

- Do I need a value, index, count, path, or boolean?
- Is the data sorted or can I sort it?
- Are duplicates allowed?
- Does contiguous matter?
- Do I need the best answer or any valid answer?
- What is repeated enough to justify a data structure?
- What complexity does the constraint force?

3. PATTERN RECOGNITION MAP

- **HashMap / HashSet:** fast lookup, complement, frequency, duplicates
- **Two pointers:** sorted arrays, opposite ends, partitioning
- **Sliding window:** contiguous subarray or substring
- **Stack / Deque:** matching pairs, next greater, monotonic scans
- **Queue / BFS:** shortest path in unweighted graph, levels
- **DFS / Recursion:** trees, graphs, backtracking
- **Binary search:** sorted data or monotonic answer space
- **Dynamic programming:** overlapping subproblems, choices
- **Greedy:** local best choice stays globally valid
- **Heap / PriorityQueue:** top K, streaming min/max
- **Union-Find:** connectivity and grouping
- **Prefix sums:** repeated range sums and counts

4. JAVA DEFAULTS

- **`HashMap<K, V>`** for value to index or count
- **`HashSet<T>`** for existence checks
- **`ArrayDeque<T>`** for stack or queue behavior
- **`PriorityQueue<T>`** for heap problems
- **`ArrayList<T>`** for dynamic result building
- Prefer arrays when size is fixed and index access matters

> Prefer `Deque` over old `Stack` in modern Java.

5. CONSTRAINT HEURISTICS

- **`n <= 100`**: `O(n^2)` may be fine
- **`n ~ 10^5`**: usually target `O(n)` or `O(n log n)`
- **Heavy repeated lookup**: use map or set
- **Sorted + threshold**: think binary search
- **All combinations**: backtracking or bitmasking
- **Best over many choices**: DP or greedy

6. INTERVIEW WORKFLOW

7. COMMON MISTAKES

1. Restate the problem.
2. Ask clarifying questions.
3. Present brute force.
4. Improve with a pattern.
5. State complexity.
6. Code cleanly.
7. Dry-run with examples.
8. Check edge cases last.

- Coding before defining input and output
- Ignoring constraints
- Jumping to optimization too early
- Choosing the wrong data structure
- Skipping small manual examples
- Forgetting empty input, boundaries, duplicates, overflow

8. PROBLEM TEMPLATE

```
Problem:
Restate in your own words

Input:
Type, size, constraints

Output:
What to return

Brute Force:
Simplest correct idea

Pattern / Better Idea:
HashMap? Sliding window? Stack? BFS? DP?

Data Structure:
Array / Map / Set / Deque / Heap / Graph

Algorithm Steps:
1. ...
2. ...
3. ...

Complexity:
Time: ...
Space: ...

Edge Cases:
Empty input, one element, duplicates, negatives, overflow
```

9. FAST EXAMPLE: TWO SUM

Problem	Return indices of two numbers whose sum equals target.
Brute Force	Check every pair: `O(n^2)`.
Better Idea	Store seen value -> index in a `HashMap`.
Why It Works	Each element asks: "Have I already seen `target - current`?"

```java
Map<Integer, Integer> seen = new HashMap<>();

for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (seen.containsKey(need)) {
        return new int[] { seen.get(need), i };
    }
    seen.put(nums[i], i);
}
```

10. EDGE-CASE CHECKLIST

empty input | **single element** | **duplicates** | **negative numbers** | **sorted vs unsorted** | **index bounds** | **overflow** | **no valid answer**

Use: understand -> choose pattern -> code -> verify